

INDEX

Week No.	Name of the Experiment	Page
1	Perform an experiment to show how the weight and bias value effects the value of neurons.	1-2
2	Perform an experiment to show how the choice of activation function effect the output of neuron experiment with purelin(n).	3-5
3	Perform an experiment to show how the choice of activation function effect the output of neuron experiment with Tansig(n).	6-7
4	Perform an experiment to show how the choice of activation function effect the output of neuron experiment with logsig(n).	8-9
5	Perform an experiment to show how the perceptron learning rule works for linearly separable problems.	10-11
6	Perform an experiment to show how the perceptron learning rule works for Non-linearly separable problems	12-14
7	Write a program to draw a graph with multiple curve.	15-16
8	Setting up the Spyder IDE Environment and Executing a Python Program.	17-20
9	Installing Keras, Tensorflow and Pytorch libraries and making use of them.	21-22
10	Train a sentiment analysis model on IMDB dataset, use RNN layers with LSTM/GRU notes.	23-24
11	Applying the Autoencoder algorithms for encoding the real-world data.	25-27
12	Applying Generative Adversial Networks for image generation and unsupervised tasks.	28-34

EXPERIMENT NO: 1

AIM : Perform an experiment to show how the weight and bias value effects the value of neurons.

ALGORITHM:

SOURCE CODE:

```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size):
        # Initialize the weights and biases for the layers

        # Input to hidden layer weights (random initialization)
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        # Hidden layer biases
        self.bias_hidden = np.zeros((1, hidden_size))

        # Hidden to output layer weights (random initialization)
        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        # Output layer biases
        self.bias_output = np.zeros((1, output_size))

        # Print the initialized weights and biases
        print("Initialized weights and biases:")
        print("Weights (Input -> Hidden):\n", self.weights_input_hidden)
        print("Biases (Hidden Layer):\n", self.bias_hidden)
        print("Weights (Hidden -> Output):\n", self.weights_hidden_output)
        print("Biases (Output Layer):\n", self.bias_output)

    def forward(self, X):
        # Forward pass through the network

        # Input to hidden layer activation (using sigmoid for example)
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.sigmoid(self.hidden_input)

        # Hidden to output layer activation
        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.sigmoid(self.output_input)

        return self.output

    def sigmoid(self, x):
```

```
return 1 / (1 + np.exp(-x))

# Example usage:
input_size = 3  # Number of input neurons
hidden_size = 5 # Number of neurons in the hidden layer
output_size = 2 # Number of output neurons

# Create the neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size)

# Example input data (for a batch of 2 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9], [0.2, 0.8, 0.6]])

# Perform a forward pass with the input data
output = nn.forward(X_input)
```

OUTPUT:**1. Initialized weights and biases:**

```
Weights (Input -> Hidden):
[[ 0.13733665  0.60730689 -0.93537005 -1.47362686 -1.37556107]
 [ 0.03513254  0.50556858 -1.04688382 -1.22800211  0.09775871]
 [-1.37266931  1.2456073   1.73657668  0.13192531 -0.48666199]]
Biases (Hidden Layer):
[[0. 0. 0. 0. 0.]]
Weights (Hidden -> Output):
[[ 1.28871665 -0.53919145]
 [ 1.33294701  0.9834711 ]
 [-2.16332141 -1.31205913]
 [ 0.71992958 -0.4048897 ]
 [-1.21746859  0.14903262]]
Biases (Output Layer): [[0. 0.]]
Output of the neural network for the input data:
[[0.40413155 0.41109537]
 [0.51571203 0.47466632]]
```

EXPERIMENT NO: 2

AIM : Perform an experiment to show how the choice of activation function effect the output of neuron experiment with purelin(n).

SOURCE CODE:

```
import numpy as np
```

```
class NeuralNetwork:
```

```
    def __init__(self, input_size, hidden_size, output_size, activation_function='sigmoid'):
```

```
        # Initialize the weights and biases
```

```
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
```

```
        self.bias_hidden = np.zeros((1, hidden_size))
```

```
        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
```

```
        self.bias_output = np.zeros((1, output_size))
```

```
        # Set the activation function
```

```
        self.activation_function = activation_function
```

```
    def forward(self, X):
```

```
        # Forward pass through the network
```

```
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
```

```
        self.hidden_output = self.apply_activation(self.hidden_input)
```

```
        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
```

```
        self.output = self.apply_activation(self.output_input)
```

```
        return self.output
```

```
    def apply_activation(self, x):
```

```
        if self.activation_function == 'sigmoid':
```

```
            return self.sigmoid(x)
```

```
        elif self.activation_function == 'relu':
```

```
            return self.relu(x)
```

```
        elif self.activation_function == 'tanh':
```

```
            return self.tanh(x)
```

```
        elif self.activation_function == 'leaky_relu':
```

```
            return self.leaky_relu(x)
```

```
        elif self.activation_function == 'softmax':
```

```
            return self.softmax(x)
```

```
elif self.activation_function == 'softplus':
    return self.softplus(x)
else:
    raise ValueError(f'Activation function {self.activation_function} not supported.')

# Activation functions
def sigmoid(self, x):
    return 1 / (1 + np.exp(-x))

def relu(self, x):
    return np.maximum(0, x)

def tanh(self, x):
    return np.tanh(x)

def leaky_relu(self, x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)

def softmax(self, x):
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True)) # Numerical stability
    return e_x / np.sum(e_x, axis=-1, keepdims=True)

def softplus(self, x):
    return np.log(1 + np.exp(x))

# Example usage:
input_size = 3 # Number of input neurons
hidden_size = 5 # Number of neurons in the hidden layer
output_size = 2 # Number of output neurons

# Create a neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size, activation_function='sigmoid')

# Example input data (for a batch of 2 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9], [0.2, 0.8, 0.6]])

# Perform a forward pass with the input data
output = nn.forward(X_input)

print("\nOutput of the neural network for the input data:")
print(output)
```

Output:

Output of the neural network for the input data:

```
[[0.6854294 0.35951906]  
 [0.70518654 0.37606834]]
```

EXPERIMENT NO: 3

AIM: Perform an experiment to show how the choice of activation function effect the output of neuron experiment with Tansig(n).

ALGORITHM**SOURCE CODE:**

```
import numpy as np

class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, activation_function='sigmoid',
loss_function='mse'):
        # Initialize the weights and biases
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
        self.bias_hidden = np.zeros((1, hidden_size))

        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
        self.bias_output = np.zeros((1, output_size))

        # Set the activation function
        self.activation_function = activation_function
        self.loss_function = loss_function

    def forward(self, X):
        # Forward pass through the network
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
        self.hidden_output = self.apply_activation(self.hidden_input)

        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
        self.output = self.apply_activation(self.output_input)

        return self.output

    def apply_activation(self, x):
        if self.activation_function == 'sigmoid':
            return self.sigmoid(x)
        elif self.activation_function == 'relu':
            return self.relu(x)
        elif self.activation_function == 'tanh':
            return self.tanh(x)
        elif self.activation_function == 'leaky_relu':
            return self.leaky_relu(x)
        elif self.activation_function == 'softmax':
            return self.softmax(x)
```

```
elif self.activation_function == 'softplus':
    return self.softplus(x)
else:
    raise ValueError(f"Activation function {self.activation_function} not supported.")

# Activation functions
def sigmoid(self, x):
    return 1 / (1 + np.exp(-x))

def relu(self, x):
    return np.maximum(0, x)

def tanh(self, x):
    return np.tanh(x)

def leaky_relu(self, x, alpha=0.01):
    return np.where(x > 0, x, alpha * x)

def softmax(self, x):
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True)) # Numerical stability
    return e_x / np.sum(e_x, axis=-1, keepdims=True)

def softplus(self, x):
    return np.log(1 + np.exp(x))

# Loss function
def compute_loss(self, y_true, y_pred):
    if self.loss_function == 'mse':
        return self.mean_squared_error(y_true, y_pred)
    elif self.loss_function == 'cross_entropy':
        return self.cross_entropy_loss(y_true, y_pred)
    else:
        raise ValueError(f"Loss function {self.loss_function} not supported.")

# Mean Squared Error (MSE)
def mean_squared_error(self, y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

# Cross-Entropy Loss (for classification)
def cross_entropy_loss(self, y_true, y_pred):
    # To avoid log(0), we add a small epsilon value
    epsilon = 1e-15
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))

# Example usage:
```



```
input_size = 3    # Number of input neurons
hidden_size = 5   # Number of neurons in the hidden layer
output_size = 2   # Number of output neurons

# Create a neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size, activation_function='sigmoid',
loss_function='mse')

# Example input data (for a batch of 2 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9], [0.2, 0.8, 0.6]])

# Actual target values (e.g., for regression, use continuous values)
y_true = np.array([[0.5, 0.2], [0.7, 0.3]])

# Perform a forward pass with the input data
output = nn.forward(X_input)

# Calculate the loss using the true values and the predicted output
loss = nn.compute_loss(y_true, output)

print("\nPredicted output:")
print(output)
print("\nLoss (MSE):")
print(loss)
```

OUTPUT :

Predicted output:

```
[[0.87654603 0.21322158]
 [0.79446709 0.24716415]]
```

Loss (MSE):

```
0.03841934510105007
```

EXPERIMENT NO: 4

AIM : Perform an experiment to show how the choice of activation function effect the output of neuron experiment with logsig(n).

SOURCE CODE:

```
import numpy as np
```

```
class NeuralNetwork:
```

```
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.01):
```

```
        # Initialize the weights and biases
```

```
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
```

```
        self.bias_hidden = np.zeros((1, hidden_size))
```

```
        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
```

```
        self.bias_output = np.zeros((1, output_size))
```

```
        # Learning rate for gradient descent
```

```
        self.learning_rate = learning_rate
```

```
    def forward(self, X):
```

```
        # Forward pass through the network
```

```
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
```

```
        self.hidden_output = self.sigmoid(self.hidden_input)
```

```
        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
```

```
        self.output = self.sigmoid(self.output_input)
```

```
        return self.output
```

```
    def sigmoid(self, x):
```

```
        return 1 / (1 + np.exp(-x))
```

```
    def sigmoid_derivative(self, x):
```

```
# Derivative of the sigmoid function
return x * (1 - x)

def compute_loss(self, y_true, y_pred):
    # Mean Squared Error (MSE) loss function
    return np.mean((y_true - y_pred) ** 2)

def backward(self, X, y_true, y_pred):
    # Calculate the loss gradient with respect to the output layer
    output_error = y_pred - y_true
    output_delta = output_error * self.sigmoid_derivative(y_pred)

    # Calculate the loss gradient with respect to the hidden layer
    hidden_error = output_delta.dot(self.weights_hidden_output.T)
    hidden_delta = hidden_error * self.sigmoid_derivative(self.hidden_output)

    # Update weights and biases using the gradients
    self.weights_hidden_output -= self.learning_rate * self.hidden_output.T.dot(output_delta)
    self.bias_output -= self.learning_rate * np.sum(output_delta, axis=0, keepdims=True)

    self.weights_input_hidden -= self.learning_rate * X.T.dot(hidden_delta)
    self.bias_hidden -= self.learning_rate * np.sum(hidden_delta, axis=0, keepdims=True)

def train(self, X, y_true, epochs=1000):
    # Train the neural network using forward and backward propagation
    for epoch in range(epochs):
        # Forward pass
        y_pred = self.forward(X)

        # Compute loss
        loss = self.compute_loss(y_true, y_pred)

        # Backward pass (backpropagation)
```

```
self.backward(X, y_true, y_pred)

# Print the loss at each epoch
if epoch % 100 == 0:
    print(f'Epoch {epoch} | Loss: {loss}')

# Example usage:
input_size = 3 # Number of input neurons
hidden_size = 5 # Number of neurons in the hidden layer
output_size = 1 # Number of output neurons (for regression, it can be 1)

# Create the neural network instance
nn = NeuralNetwork(input_size, hidden_size, output_size, learning_rate=0.01)

# Example input data (for a batch of 4 samples with 3 features each)
X_input = np.array([[0.1, 0.5, 0.9],
                    [0.2, 0.8, 0.6],
                    [0.9, 0.1, 0.4],
                    [0.5, 0.3, 0.8]])

# Actual target values (e.g., for regression)
y_true = np.array([[0.5], [0.7], [0.2], [0.9]])

# Train the neural network
nn.train(X_input, y_true, epochs=1000)

# After training, let's test the network with the same input data
y_pred = nn.forward(X_input)

print("\nPredicted output after training:")
print(y_pred)
```

OUTPUT:

Epoch 0 | Loss: 0.07261206704099438
Epoch 100 | Loss: 0.06947570558090783
Epoch 200 | Loss: 0.06782901320496823
Epoch 300 | Loss: 0.06695239016938156
Epoch 400 | Loss: 0.06645255161349309
Epoch 500 | Loss: 0.06613132067718504
Epoch 600 | Loss: 0.0658932961768156
Epoch 700 | Loss: 0.06569374647874232
Epoch 800 | Loss: 0.0655120134274131
Epoch 900 | Loss: 0.06533863058660963
Predicted output after training:
[[0.5828925]
[0.59719375]
[0.5631467]
[0.56629168]]

EXPERIMENT NO: 5

AIM: Perform an experiment to show how the perceptron learning rule works for linearly separable problems.

SOURCE CODE:

```
import numpy as np
```

```
class NeuralNetwork:
```

```
    def __init__(self, input_size, hidden_size, output_size, learning_rate=0.1):
```

```
        # Initialize weights and biases
```

```
        self.weights_input_hidden = np.random.randn(input_size, hidden_size)
```

```
        self.bias_hidden = np.zeros((1, hidden_size))
```

```
        self.weights_hidden_output = np.random.randn(hidden_size, output_size)
```

```
        self.bias_output = np.zeros((1, output_size))
```

```
        # Learning rate for gradient descent
```

```
        self.learning_rate = learning_rate
```

```
    def forward(self, X):
```

```
        # Forward pass: Calculate the activations for hidden and output layers
```

```
        self.hidden_input = np.dot(X, self.weights_input_hidden) + self.bias_hidden
```

```
        self.hidden_output = self.sigmoid(self.hidden_input)
```

```
        self.output_input = np.dot(self.hidden_output, self.weights_hidden_output) + self.bias_output
```

```
        self.output = self.sigmoid(self.output_input)
```

```
        return self.output
```

```
    def sigmoid(self, x):
```

```
        # Sigmoid activation function
```

```
        return 1 / (1 + np.exp(-x))
```

```
    def sigmoid_derivative(self, x):
```

```
        # Derivative of the sigmoid function
```

```
        return x * (1 - x)
```

```
    def compute_loss(self, y_true, y_pred):
```

```
        # Mean Squared Error loss function
```

```
        return np.mean((y_true - y_pred) ** 2)
```

```
    def backward(self, X, y_true, y_pred):
```

```
        # Backward pass: Calculate the gradients and update weights using gradient descent
```

```
        # Calculate output layer error and delta
```

```
output_error = y_pred - y_true
output_delta = output_error * self.sigmoid_derivative(y_pred)

# Calculate hidden layer error and delta
hidden_error = output_delta.dot(self.weights_hidden_output.T)
hidden_delta = hidden_error * self.sigmoid_derivative(self.hidden_output)

# Update weights and biases using the gradients
self.weights_hidden_output -= self.learning_rate * self.hidden_output.T.dot(output_delta)
self.bias_output -= self.learning_rate * np.sum(output_delta, axis=0, keepdims=True)

self.weights_input_hidden -= self.learning_rate * X.T.dot(hidden_delta)
self.bias_hidden -= self.learning_rate * np.sum(hidden_delta, axis=0, keepdims=True)

def train(self, X, y_true, epochs=10000):
    # Train the neural network using forward and backward propagation
    for epoch in range(epochs):
        # Forward pass
        y_pred = self.forward(X)

        # Compute loss
        loss = self.compute_loss(y_true, y_pred)

        # Backward pass (backpropagation)
        self.backward(X, y_true, y_pred)

        # Print loss every 1000 epochs
        if epoch % 1000 == 0:
            print(f"Epoch {epoch} | Loss: {loss}")

def test(self, X):
    # Test the neural network with the input data
    return self.forward(X)

# Example usage for the XOR problem:
# Input data for XOR (4 examples, 2 features each)
X_input = np.array([[0, 0],
                    [0, 1],
                    [1, 0],
                    [1, 1]])

# Output data for XOR (4 examples, 1 output each)
y_true = np.array([[0], [1], [1], [0]])

# Create a neural network instance
nn = NeuralNetwork(input_size=2, hidden_size=4, output_size=1, learning_rate=0.1)
```

```
# Train the neural network
print("Training the neural network...")
nn.train(X_input, y_true, epochs=10000)

# After training, test the neural network on the XOR input
print("\nTesting the neural network on the XOR input:")
y_pred = nn.test(X_input)

print("\nPredicted output after training:")
print(y_pred)
```

OUTPUT:

```
Training the neural network...
Epoch 0 | Loss: 0.2526436661693838
Epoch 1000 | Loss: 0.2397158667257264
Epoch 2000 | Loss: 0.13062995557194468
Epoch 3000 | Loss: 0.02872194228940983
Epoch 4000 | Loss: 0.011393299508316048
Epoch 5000 | Loss: 0.006560568046749035
Epoch 6000 | Loss: 0.00447809366050457
Epoch 7000 | Loss: 0.0033542019595272027
Epoch 8000 | Loss: 0.002661533382957928
Epoch 9000 | Loss: 0.00219593198121231

Testing the neural network on the XOR input:

Predicted output after training:
[[0.02415875]
 [0.95222034]
 [0.95940909]
 [0.05421303]]
```


EXPERIMENT NO: 6

AIM : Perform an experiment to show how the perceptron learning rule works for Non-linearly separable problems.

Source code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load the dataset
file_path = "/content/sample_data/heart_disease.csv" # Update this with the correct path
data = pd.read_csv(file_path)

# Assuming the target column is named "target" and features are numerical
X = data.drop(columns=["target"]).values
y = data["target"].values

# Preprocess the target variable
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y)
y = to_categorical(y) # Convert to one-hot encoding for classification

# Preprocess the features
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Reshape data for CNN input (CNN expects a 3D input: samples, timesteps, features)
X = np.expand_dims(X, axis=2) # Add a dimension for features

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Build the CNN model
model = Sequential([
    Conv1D(filters=32, kernel_size=3, activation='relu', input_shape=(X.shape[1], 1)),
```

```

MaxPooling1D(pool_size=2),
Dropout(0.3),
Conv1D(filters=64, kernel_size=3, activation='relu'),
MaxPooling1D(pool_size=2),
Dropout(0.3),
Flatten(),
Dense(128, activation='relu'),
Dropout(0.3),
Dense(y.shape[1], activation='softmax') # Adjust for the number of classes
])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2,
verbose=1)

# Evaluate the model
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')

# Save the model
model.save("heart_disease_cnn_model.h5")

```

OUTPUT:

```

Epoch
- val_loss: 0.6508
Epoch 10/20
7/7 -----0s 14ms/step - accuracy: 0.5485 - loss: 0.6771 - val_accuracy:
0.6939 - val_loss: 0.6536
Epoch 1/20
7/7 -----2s 57ms/step - accuracy: 0.3981 - loss: 0.7163 - val_accuracy:
0.5306 - val_loss: 0.6952
Epoch 2/20
7/7 -----0s 12ms/step - accuracy: 0.6080 - loss: 0.6850 - val_accuracy:
0.5918 - val_loss: 0.6914
Epoch 3/20
7/7 -----0s 12ms/step - accuracy: 0.5316 - loss: 0.6904 - val_accuracy:
0.6327 - val_loss: 0.6823
Epoch 4/20
7/7 -----0s 12ms/step - accuracy: 0.5459 - loss: 0.6858 - val_accuracy:
0.6531 - val_loss: 0.6750
Epoch 5/20
7/7 -----0s 12ms/step - accuracy: 0.6064 - loss: 0.6799 - val_accuracy:
0.6327 - val_loss: 0.6690
Epoch 6/20
7/7 -----0s 14ms/step - accuracy: 0.5923 - loss: 0.6736 - val_accuracy:

```

0.6531 - val_loss: 0.6631

Epoch 7/20
7/7  0s 13ms/step - accuracy: 0.5802 - loss: 0.6710 -
val_accuracy: 0.6327 - val_loss: 0.6576

Epoch 8/20
7/7  0s 12ms/step - accuracy: 0.6696 - loss:

0.6485 - val_accuracy: 0.6939 - val_loss: 0.6545

Epoch 9/20

7/7  0s 15ms/step - accuracy: 0.6508 - loss: 0.6444 -
val_accuracy: 0.6939

Epoch 11/20

7/7  0s 14ms/step - accuracy: 0.7071 - loss: 0.6336
- val_accuracy: 0.6939 - val_loss: 0.6469

Epoch 12/20

7/7  0s 11ms/step - accuracy: 0.6538 - loss:
0.6388 - val_accuracy: 0.6939 - val_loss: 0.6394

Epoch 13/20

7/7  0s 12ms/step - accuracy:
0.6701 - loss: 0.6348 - val_accuracy: 0.6735 - val_loss: 0.6327

Epoch 14/20

7/7  0s 11ms/step - accuracy: 0.6995 - loss:
0.6027 - val_accuracy: 0.6735 - val_loss: 0.6265

Epoch 15/20

7/7  0s 7ms/step - accuracy: 0.6278 - loss:

0.6220 - val_accuracy: 0.6735 - val_loss: 0.6184

Epoch 16/20

7/7  0s 10ms/step - accuracy: 0.6609 - loss:

0.6122 - val_accuracy: 0.5918 - val_loss: 0.6339

Epoch 17/20

7/7  0s 9ms/step - accuracy: 0.7014 - loss:

0.5868 - val_accuracy: 0.5918 - val_loss: 0.6480

Epoch 18/20

7/7  0s 10ms/step - accuracy: 0.6149 - loss:

0.6221 - val_accuracy: 0.6939 - val_loss: 0.6028

Epoch 19/20

7/7  0s 7ms/step - accuracy: 0.6731 - loss:

0.5685 - val_accuracy: 0.6735 - val_loss: 0.5988

Epoch 20/20

7/7  0s 10ms/step - accuracy: 0.7165 - loss:

0.5732 - val_accuracy: 0.7143 - val_loss: 0.5899

Test Accuracy: 59.02%

EXPERIMENT :7

AIM : Write a program to draw a graph with multiple curve.

SOURCE CODE:

```
import tensorflow as tf
from tensorflow.keras import layers, models, datasets
import matplotlib.pyplot as plt

# Load and preprocess the dataset (using CIFAR-10 as an example)
def load_data():
    (x_train, y_train), (x_test, y_test) = datasets.cifar10.load_data()
    x_train, x_test = x_train / 255.0, x_test / 255.0 # Normalize to [0, 1]
    y_train = tf.keras.utils.to_categorical(y_train, 10) # One-hot encoding
    y_test = tf.keras.utils.to_categorical(y_test, 10)
    return (x_train, y_train), (x_test, y_test)

# Define the CNN model
def create_model():
    model = models.Sequential()
    model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.MaxPooling2D((2, 2)))
    model.add(layers.Conv2D(64, (3, 3), activation='relu'))
    model.add(layers.Flatten())
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(10, activation='softmax')) # Output layer for 10 classes
    return model

# Compile the model
def compile_model(model):
    model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

# Train the model
def train_model(model, x_train, y_train, x_test, y_test):
    history = model.fit(x_train, y_train, epochs=10,
                        validation_data=(x_test, y_test), batch_size=64)
    return history

# Visualize training performance
def plot_history(history):
```

```

plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0, 1])
plt.legend(loc='lower right')
plt.show()

# Main function
if __name__ == "__main__":
    # Load data
    (x_train, y_train), (x_test, y_test) = load_data()

    # Create the CNN model
    model = create_model()

    # Compile the model
    compile_model(model)

    # Train the model
    history = train_model(model, x_train, y_train, x_test, y_test)

    # Evaluate the model
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    print(f"Test accuracy: {test_acc:.2f}")

    # Plot training history
    plot_history(history)

```

OUTPUT:

```

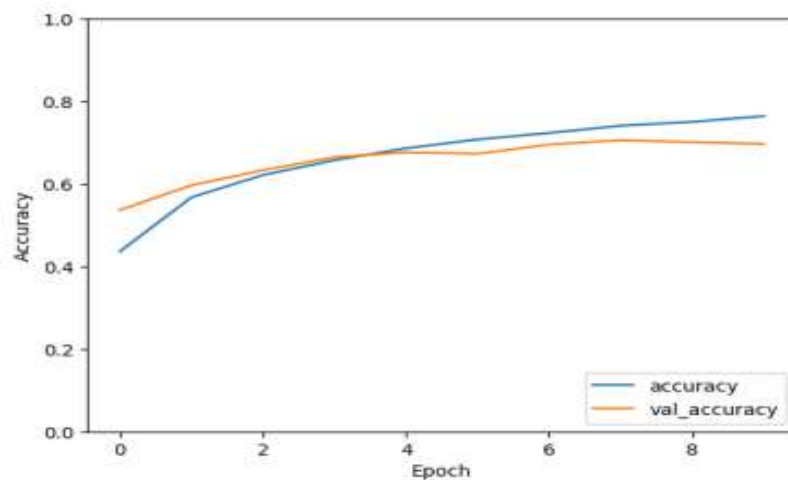
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 2s 0us/step
/usr/local/lib/python3.10/dist-packages/keras/src/layers/convolutional/base_conv.py:107:
UserWarning: Do not pass an
    input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an
`Input(shape)`
    object as the first layer in the model instead.
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
782/782 ————— 70s 87ms/step - accuracy: 0.3473 - loss: 1.7702 -
val_accuracy: 0.5366 -
    val_loss: 1.2916
Epoch 2/10
782/782 ————— 82s 87ms/step - accuracy: 0.5529 - loss: 1.2518 -
val_accuracy: 0.5964 -
    val_loss: 1.1262
Epoch 3/10
782/782 ————— 85s 90ms/step - accuracy: 0.6156 - loss: 1.0826 -
val_accuracy: 0.6333 -

```

```

val_loss: 1.0362
Epoch 4/10
782/782 ————— 69s 89ms/step - accuracy: 0.6527 - loss: 0.9867 -
val_accuracy: 0.6649 -
val_loss: 0.9496
Epoch 5/10
782/782 ————— 85s 92ms/step - accuracy: 0.6828 - loss: 0.9037 -
val_accuracy: 0.6763 -
val_loss: 0.9290
Epoch 6/10
782/782 ————— 79s 89ms/step - accuracy: 0.7051 - loss: 0.8456 -
val_accuracy: 0.6732
val_loss: 0.9262
Epoch 7/10
782/782 ————— 80s 87ms/step - accuracy: 0.7223 - loss: 0.7911
- val_accuracy: 0.6955
val_loss: 0.8803
Epoch 8/10
782/782 ————— 83s 88ms/step - accuracy: 0.7454 - loss:
0.7455 - val_accuracy: 0.7060
val_loss: 0.8540
Epoch 9/10
782/782 ————— 81s 87ms/step - accuracy: 0.7550 - loss:
0.7026 - val_accuracy: 0.7010
- val_loss: 0.8686
Epoch 10/10
782/782 ————— 85s 91ms/step - accuracy: 0.7669 - loss: 0.6697
- val_accuracy: 0.6972 -
val_loss: 0.8975
313/313 - 3s - 11ms/step - accuracy: 0.6972 - loss: 0.8975
Test accuracy: 0.70

```



EXPERIMENT :8

AIM: Setting up the Spyder IDE Environment and Executing a Python Program.

SOURCE CODE:

```
import pandas as pd

import numpy as np

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler, LabelEncoder

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense, Dropout

from tensorflow.keras.utils import to_categorical


# Load the dataset

file_path = "/content/sample_data/heart_disease.csv" # Update this with your dataset's file path

data = pd.read_csv(file_path)


# Assuming the target column is named "target" and features are numerical

X = data.drop(columns=["target"]).values

y = data["target"].values


# Preprocess the target variable

label_encoder = LabelEncoder()

y = label_encoder.fit_transform(y)

y = to_categorical(y) # Convert to one-hot encoding for classification
```

```
# Preprocess the features
```

```
scaler = StandardScaler()
```

```
X = scaler.fit_transform(X)
```

```
# Reshape data for RNN input (RNN expects a 3D input: samples, timesteps, features)
```

```
X = np.expand_dims(X, axis=1) # Add a time dimension
```

```
# Split the dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Build the RNN model
```

```
model = Sequential([
```

```
    LSTM(64, activation='tanh', return_sequences=True, input_shape=(X.shape[1], X.shape[2])),
```

```
    Dropout(0.3),
```

```
    LSTM(32, activation='tanh'),
```

```
    Dropout(0.3),
```

```
    Dense(128, activation='relu'),
```

```
    Dropout(0.3),
```

```
    Dense(y.shape[1], activation='softmax') # Adjust for the number of classes
```

```
])
```

```
# Compile the model
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
```

```
# Train the model
```

```
history = model.fit(X_train, y_train, epochs=20, batch_size=32, validation_split=0.2, verbose=1)
```

```
# Evaluate the model
```

```
test_loss, test_accuracy = model.evaluate(X_test, y_test, verbose=0)
```

```
print(f'Test Accuracy: {test_accuracy * 100:.2f}%')
```

```
# Save the model
```

```
model.save("heart_disease_rnn_model.h5")
```

OUTPUT:

```
Epoch 1/20
/usr/local/lib/python3.10/dist-packages/keras/src/layers/rnn/rnn.py:204: UserWarning: Do not pass an
`input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
super().__init__(**kwargs)
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 89ms/step - accuracy: 0.6326 - loss: 0.6909 -
val_accuracy: 0.7551 -
val_loss: 0.6797
Epoch 2/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.6951 - loss: 0.6819 -
val_accuracy: 0.7347 -
val_loss: 0.6672
Epoch 3/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.7393 - loss: 0.6709 -
val_accuracy: 0.8367 -
val_loss: 0.6516
Epoch 4/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8300 - loss: 0.6518 -
val_accuracy: 0.8367 -
val_loss: 0.6336
Epoch 5/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.8382 - loss: 0.6336 -
val_accuracy: 0.8571 -
val_loss: 0.6110
Epoch 6/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.8269 - loss: 0.6146 -
val_accuracy: 0.8571 -
val_loss: 0.5793
Epoch 7/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8707 - loss: 0.5787 -
val_accuracy: 0.8571 -
val_loss: 0.5366
Epoch 8/20
7/7 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.8612 - loss: 0.5197 -
val_accuracy: 0.8571 -
```

```

val_loss: 0.4886
Epoch 9/20
7/7 -----0s 14ms/step - accuracy: 0.8397 - loss: 0.4870 - val_accuracy:
0.8571 -
val_loss: 0.4586

Epoch 10/20
7/7 -----0s 17ms/step - accuracy: 0.8460 - loss: 0.4421 - val_accuracy:
0.8367 -
val_loss: 0.4361
Epoch 11/20
7/7 -----0s 25ms/step - accuracy: 0.8611 - loss: 0.4175 -
val_accuracy: 0.8571 -
val_loss: 0.4101
Epoch 12/20
7/7 -----0s 16ms/step - accuracy: 0.8663 - loss: 0.3781 - val_accuracy:
0.8571 -
val_loss: 0.3930
Epoch 13/20
7/7 -----0s 14ms/step - accuracy: 0.8723 - loss: 0.3492 - val_accuracy:
0.8571 -
val_loss: 0.3814
Epoch 14/20
7/7 -----0s 14ms/step - accuracy: 0.8477 - loss: 0.4125 -
val_accuracy: 0.8571 -
val_loss: 0.3775
Epoch 15/20
7/7 -----0s 9ms/step - accuracy: 0.8768 - loss: 0.3225 - val_accuracy:
0.8367 -
val_loss: 0.3859
Epoch 16/20
7/7 -----0s 14ms/step - accuracy: 0.8781 - loss: 0.3256 -
val_accuracy: 0.8163 -
val_loss: 0.3909
Epoch 17/20
7/7 -----0s 11ms/step - accuracy: 0.8708 - loss: 0.3154 -
val_accuracy: 0.8367 -
val_loss: 0.3893
Epoch 18/20
7/7 -----0s 9ms/step - accuracy: 0.8408 - loss: 0.3696 - val_accuracy:
0.8571 -
val_loss: 0.3845
Epoch 19/20
7/7 -----0s 11ms/step - accuracy: 0.8565 - loss: 0.3285 -
val_accuracy: 0.8571 -
val_loss: 0.3792
Epoch 20/20
7/7 -----0s 9ms/step - accuracy: 0.8575 - loss: 0.3406 - val_accuracy:
0.8571 -
val_loss: 0.3810
Test Accuracy: 85.25%

```

EXPERIMENT 9:

AIM : Installing Keras, Tensorflow and Pytorch libraries and making use of them.

SOURCE CODE:

```
import cv2

import matplotlib.pyplot as plt

# Load pre-trained Haar Cascade classifier for face detection

haarcascade_path = cv2.data.haarcascades + 'haarcascade_frontalface_default.xml'

face_cascade = cv2.CascadeClassifier(haarcascade_path)

def detect_faces(image_path):

    """Detect faces in an image using Haar Cascade."""

    # Load the image

    image = cv2.imread(image_path)

    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY) # Convert to grayscale

    # Detect faces

    faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))

    # Draw rectangles around detected faces

    for (x, y, w, h) in faces:

        cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

    # Convert the image to RGB for displaying
```

```
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Display the image
```

```
plt.imshow(image_rgb)
```

```
plt.axis("off")
```

```
plt.show()
```

```
print(f"Detected {len(faces)} face(s).")
```

```
# Test the program with an example image
```

```
image_path = "/content/sample_data/IMG-20240802-WA0134.jpg" # Replace with the path to your image
```

```
detect_faces(image_path)
```

OUTPUT:

